

Testing

Section	Part	Timestamp
A) Face Recognition System	1. Capture live video frames from the camera using OpenCV's cv2.VideoCapture(0) (adaptable to Picamera2 on Raspberry Pi deployment)	
	2. Detect all faces in each frame using MTCNN (Multi-Task Cascaded Convolutional Network), which returns bounding box coordinates (x1, y1, x2, y2) for each detected face	
	3. Clamp bounding box coordinates to image bounds and skip any detected face region smaller than 20x20 pixels to avoid processing noise or partial detections	
	4. Extract each face region and pass it through FaceNet (InceptionResnetV1 pretrained on VGGFace2) , which converts the face into a 512-dimensional embedding vector — a compact numerical representation of facial features learned through deep learning	
	5. Compare the live embedding against each stored mean embedding (generated during the upload/enrolment process) using cosine similarity (<code>torch.cosine_similarity</code>), which produces a similarity score between 0 and 1	
	6. Select the known person with the highest cosine similarity score as the best match	
	7. If the best cosine similarity score exceeds the threshold of 0.75 , display the person's name alongside the confidence score (e.g., "Rishi 0.89") and draw a green bounding box around the face, indicating the person is recognised	
	8. If the best cosine similarity score falls below 0.75 , label the person as "Unknown" and draw a red bounding box , indicating the person is not recognised by the system	
	9. Process multiple faces independently within a single frame, each receiving their own label and bounding box	

Section	Part	Timestamp
B) Upload Image System	1. Allow the user to upload multiple images of a new person through a file dialog (<code>tkinter.filedialog.askopenfilename</code> s)	
	2. Accept only images in .jpg or .png format as enforced by the file type filter	
	3. Require the user to enter the person's name before submission	
	4. Open each uploaded image using PIL , convert to RGB , and pass through MTCNN to detect and crop the face	
	5. Skip any image where no face is detected by MTCNN (i.e., <code>img_tensor</code> is <code>None</code>)	
	6. Generate a 512-dimensional embedding for each successfully detected face using FaceNet (InceptionResnetV1)	
	7. Calculate the mean (average) embedding across all uploaded images by stacking all embedding tensors and computing <code>mean(dim=0)</code> — this produces a single representative embedding that is more robust than any individual image	
	8. If an embedding already exists for the same person name under the same user, delete the old entry and replace it with the new mean embedding	
	9. Store the mean embedding in the embedding_table in PostgreSQL, linked to the user_id and person_name	
C) Database System	1. Store all detection events in the detection_history table, recording: ◦ <code>user_id</code> — the logged-in user who owns this detection ◦ <code>person_name</code> — the detected person's name or "Unknown" ◦ <code>is_known</code> — boolean indicating whether the person was recognised ◦ <code>confidence_score</code> — the cosine similarity score (0.0 for unknown faces) ◦ <code>detected_at</code> — timestamp of when the detection occurred	

Section	Part	Timestamp
	2. Store face enrolment data in the embedding_table , containing: ◦ <code>user_id</code> — the user who enrolled this person ◦ <code>person_name</code> — the enrolled person's name ◦ <code>embedding</code> — the 512-dimensional mean embedding vector stored as an array	
	3. Allow users to view the Manage Faces table showing each enrolled person's name, number of embeddings, and user ID	
	4. Allow users to remove a selected person from the facial recognition database, deleting their embedding from <code>embedding_table</code>	
	5. Filter all database queries by <code>user_id</code> so that each user can only see and manage their own data	
D) Historical Data System	1. Implement a 30-second cooldown window per detected person to prevent duplicate entries flooding the database — if the same person (by <code>person_name</code>) was logged within the last 30 seconds, skip the insert	
	2. Display detection history in a Tkinter Treeview table with columns: Person Name, Known (Yes/No), Confidence, Detected At	
	3. Sort historical data by detected_at DESC (most recent first)	
	4. Format the confidence score to 2 decimal places and the timestamp as YYYY-MM-DD HH:MM:SS	
	5. Provide a Refresh button to reload the latest data from the database	
	6. Provide a Delete History button that clears all detection history for the current user and refreshes the table	
E) User Authentication System	1. Require users to create an account with a username and password through the Create New Account form	
	2. Require users to enter the password twice to confirm it matches before account creation	

Section	Part	Timestamp
	3. Validate that neither username nor password is empty — display an error if either field is blank	
	4. Hash the password using bcrypt (<code>bcrypt.hashpw</code> with <code>bcrypt.gensalt()</code>) before storing in the database — no plaintext passwords are ever saved	
	5. Store the hashed password in the login_details table alongside the <code>user_id</code> (auto-generated primary key) and username	
	6. During login, retrieve the stored hash for the entered username and verify using bcrypt.checkpw — comparing the entered password's hash against the stored hash	
	7. Return an error message "Invalid Username or Password" if the username does not exist or the password does not match	
	8. Allow users to change their password by entering the old password (verified via <code>bcrypt</code>) and the new password twice	
	9. Allow users to delete their account , which cascades deletion across <code>detection_history</code> , <code>embedding_table</code> , and <code>login_details</code> (in that order) after password confirmation	
	10. Provide a "See Password" checkbox on all password entry forms to toggle password visibility	
F) User Interface System	1. Provide a graphical user interface built entirely with Tkinter — no command-line interaction required	
	2. Start with a Login Page containing username/ password fields, a Submit button, and a Create New Account button	
	3. Display an ethical Terms and Agreement confirmation before new account creation, requiring the user to click Continue or Cancel	
	4. After login, show a Home Page with the user's name and buttons for: ◦ Live Camera — opens the live face recognition feed ◦ Upload Images — opens the image upload/enrolment form	

Section	Part	Timestamp
	◦ Database — opens the database management menu ◦ Account (icon) — opens account settings (Change Password, Delete Account) ◦ Logout (icon) — returns to the Login Page	
	5. Implement a window navigation system where each page opens as a <code>TopLevel</code> window, the previous window is hidden (<code>withdraw</code>), and closing/backing returns to the previous window (<code>deiconify</code>)	
	6. Use WM_DELETE_WINDOW protocol handlers on all windows to ensure proper navigation when the user closes a window via the OS close button	
G) Hardware Interaction System	1.The system shall capture video using Picamera2 when deployed on the Raspberry Pi (currently uses OpenCV <code>videoCapture</code> for development)	
	2.The LCD screen shall display the detected person's name and confidence score in real time	
	3. LED indicators shall respond based on detection results: • Green LED for recognised (known) person • Red LED for unrecognised (unknown) person	
	4. GPIO pins shall be controlled via an appropriate library (e.g., <code>RPi.GPIO</code> or <code>gpiozero</code>)	
H) Offline Operation System	1. The system shall operate entirely offline with no internet connection required after initial setup	
	2. All face detection (MTCNN) and recognition (FaceNet) processing occurs locally using pre-downloaded PyTorch models (<code>pretrained='vggface2'</code>)	
	3. All data is stored in a local PostgreSQL database (<code>localhost</code>) — no cloud database services are used	
	4. No external API calls are made during operation — all computation runs on-device	